第1回：LLMイントロダクション

LLMアプリケーション開発 2026

外部の資料を使って答えるアプリを作る

今日のゴール

- LLMが文章を生成する仕組みを、自分の言葉で説明する
- モデル、チャットサービス、APIの違いを説明する
- 資料を渡す理由と、回答を確かめる必要性を理解する

今日はコードの実装を求めません。完成品を動かして観察します。

API利用時は毎回、起動するターミナルで：
`export OPENAI_API_KEY=sk-********`
終了時は停止して `unset OPENAI_API_KEY` 。

今日の流れ

内容
授業の全体像と前の授業とのつながり
LLMの仕組みと使い方
リポジトリのForkとCodespacesの準備
完成アプリを動かし、同じ質問を2つの方式で比較
結果の共有・確認問題・提出

観察では、回答だけでなく、LLMに渡した資料も見ます。

13回の授業

回	内容
1〜3	LLMの基礎、API、Webアプリとの連携
4〜6	外部資料、キーワード検索、意味検索
7〜8	関数の呼び出し、評価、安全な運用
9〜13	自分の題材で企画・制作・改善

完成版のアプリを5つ用意しています。毎回、その1つを読んで動かします。

毎回の進め方（第2回から）

段階	すること
1	`sample-app/` の完成版を動かす
2	コードの説明を聞く
3	`sessionNN/exercise/` の穴あきコードを埋めて動かす
4	コミットしてプッシュする
5	GitHubのURLをフォームで提出する

穴あきコードの答えは、`sample-app/` の完成版です。

前の授業とのつながり

既習の技術	今回の使い道
HTML・CSS・JavaScript	質問フォームと回答の画面
FastAPI・JSON・fetch	ブラウザとサーバーの通信
SQLite・SQL	参照資料と制作の進捗の保存
入力検証・textContent	不正な入力や出力への対策

新しく加えるのは、外部のLLM APIと知識の検索です。

LLMとは

Large Language Model：大規模言語モデル。

大量の文章などから言葉の関係を学習し、入力された文脈に続く
トークンを予測して文章を生成します。

質問への回答、要約、分類、文章の書き換えなどに利用できます。
自然な文章であることと、内容が正しいことは別に確認します。

言語モデルの簡単な歴史

時期	主な考え方
従来	直前の単語の並びから、次の単語の頻度を推定
2000〜2010年代	ニューラルネットワークで言葉の特徴を学習
2017年	Transformerを提案した論文が発表
2018年以降	大規模な事前学習を使うモデルが発展
2022年以降	対話形式のサービスが普及

Transformerの考え方

同じ「はし」でも、周りの言葉によって意味が変わります。

- 「料理を、はしを使って食べる」→「料理」「食べる」が手がかりになり、**箸**とわかる
- 「はしを渡って、川の向こうへ行く」→「渡って」「川」が手がかりになり、**橋**とわかる

Attentionは、文脈内の要素どうしの関係を計算する仕組みです。
この例では、「はし」と周りの言葉との関係が、意味を捉える手がかりになります。

この授業では数式や学習処理の実装には立ち入りません。
入力にどんな文脈を与えるかに注目します。

トークンと生成

トークンは、モデルが扱う文章の区切りです。
1文字や1単語と必ず一致するわけではありません。

1. 入力をトークンに分ける
2. 文脈から次のトークンの候補を計算する
3. 選んだトークンを文脈に加える
4. 終了条件まで繰り返す

「文章の続きを作る」仕組みを、質問への応答にも使います。

生成の途中で増える文脈

説明用に「今日は」「晴れ」「です」を順に選んだとします。

段階	次の候補を考えるための文脈
最初	入力された質問や指示
1回選択後	入力 ＋「今日は」
2回選択後	入力 ＋「今日は晴れ」
3回選択後	入力 ＋「今日は晴れです」

区切りは説明用です。実際のトークン分割や生成結果ではありません。
生成済みの内容も、次を生成する文脈に含まれます。

学習と推論

処理	何をするか	この授業
学習	データからモデルのパラメータを調整する	実施しない
推論	既に学習したモデルに入力を渡し、出力を得る	APIで利用

授業資料を質問に添付するだけでは、モデルを再学習しません。
今回の回答で参照する情報として渡します。

LLMは前の会話を覚えていない

モデルは、今回の入力だけを見て出力を作ります。

入力	出力
「私の名前は佐藤です。」	「こんにちは、佐藤さん。」
「私の名前は？」	「分かりません。」

チャット画面で前の話を踏まえて答えられるのは、
アプリがこれまでの会話を毎回まとめて送っているからです。

今日の観察1で、この仕組みを確かめます。

資料を添えた質問の例

説明用の架空資料：「図書室の開室時間は平日16時までです。」

今回送るもの	内容
参考資料	上の開室時間の案内
質問	図書室は何時まで開いている？
回答を確かめる基準	平日16時という条件が一致するか

資料は、必要になるたびにアプリが入力に含めます。
モデルが資料を覚えるわけではありません。

モデル・サービス・API

言葉	役割	例
モデル	入力から出力を生成する仕組み	GPT、Claude、Geminiなど
サービス	人が画面から利用する製品	チャット画面など
API	プログラムから利用する窓口	自作アプリからの呼び出し

画面にある検索・履歴・ファイル機能を、モデル自体の能力と混同しないことが大切です。

自作アプリが用意する機能

「質問に答える画面」を作るには、生成以外の処理も必要です。

利用者から見える機能	アプリが行う処理
質問の入力欄	入力を受け取り、長さを確認する
前の会話を踏まえた回答	会話の履歴をためて、毎回まとめて送る
資料を使った回答	資料を探して入力に含める
出典の表示	渡した本文と資料IDを表示する
失敗したときの案内	エラーを利用者向けの表示に変える

完成品で、これらが画面のどこにあるかを観察します。

代表的なモデル系列

系列	提供元	観察する点
GPT	OpenAI	APIの機能、用途ごとのモデル
Claude	Anthropic	APIの機能、用途ごとのモデル
Gemini	Google	入出力の種類、APIの機能

世代、利用条件、対応機能は変わります。公式資料で確認します。
順位を暗記するのではなく、用途に合わせて比べます。

この授業で使うモデル

文章生成と関数選択には **gpt-6-luna** を使います。
第2回からResponses APIで呼び出します。

第6回の文章のベクトル化には、別用途の
text-embedding-3-small を使います。

生成用モデルとEmbedding用モデルは、仕事が異なります。

得意なことと、確認が必要なこと

使い方	確認すること
長い文の要約	大切な条件を落としていないか
問い合わせの分類	あいまいな例をどう扱うか
コードの説明	実際のコードと一致しているか
授業資料の回答	資料に根拠があるか

もっともらしい誤答を、ハルシネーションと呼ぶことがあります。

回答の確かめ方

説明用の架空資料：「提出は金曜17時まで。形式はPDF。」

回答例	本文と照合した結果
金曜17時までにPDFで提出	日時と形式が一致
金曜中に提出	17時という条件が抜けている
土曜17時までにPDFで提出	日付が資料と一致しない

同じように自然な文章でも、資料との一致は別に調べられます。
この照合を、完成アプリの観察でも行います。

モデルが知らない情報

「この授業で使うモデルは何？」

モデルが一般的なプログラミング知識を持っていても、
この授業の資料を知っているとは限りません。

- 資料を渡す
- 必要な情報をDBから取得する
- 根拠がないときは確認できないと伝える

このための仕組みをWebアプリ側に作ります。

アプリの役割分担

場所	仕事
ブラウザ	質問を送信し、結果を表示
FastAPI	入力を確認し、資料や関数を選ぶ
SQLite	資料や進捗を保存
LLM API	渡された内容から回答や関数の引数を生成

LLMへ渡す情報と、実行できる操作をアプリ側が決めます。

授業で使う5つのアプリ

フォルダ	アプリ	学ぶこと	回
llm-call/	LLM呼び出しの基本	1回だけ呼ぶ、履歴をのせて呼ぶ	2
chat-app/	チャットアプリ	履歴をブラウザに持ち、毎回まとめて送る	1・3
keyword-app/	キーワード検索アプリ	資料を探して、それをもとに答える	1・4・5・8
vector-app/	ベクトル検索アプリ	意味の近い資料を探す	6・8
progress-app/	制作の進捗アプリ	LLMが関数を選び、PythonがDBを読む	7

5つのフォルダは互いに独立していて、それぞれ単独で動きます。

準備1：リポジトリをFork

1. 授業のリポジトリをGitHubで開く
2. 右上の「Fork」を押し、自分のアカウントにコピーを作る
3. 自分のリポジトリのURLを控える

以後の提出は、自分のリポジトリのURLで行います。
前の授業と同じ流れです。

準備2：Codespacesを起動

1. 自分のリポジトリで「Code」→「Codespaces」→「Create codespace on main」
2. 初回は数分待つ。`.venv` と必要なパッケージが自動で入る
3. 「ターミナル」→「新しいターミナル」を開く

```
source .venv/bin/activate
python --version
```

ブラウザ版のVS Codeで、前の授業と同じように作業します。

準備3：APIキーを設定

講師が配るキーを、アプリを起動するターミナルで設定します。

```
export OPENAI_API_KEY=sk-********
```

- 同じターミナルから起動する。新しいターミナルでは再設定する
- キーをファイル、Codespaces Secrets、提出物に書かない
- 終了時はサーバーを停止し、`unset OPENAI_API_KEY`

質問するたびにAPI利用料金が発生します。

完成アプリの起動：チャットアプリ

```
cd sample-app/chat-app
python main.py
```

「ポート」タブから8000番を開きます。可視性はPrivateのままにします。
終了はCtrl+C。ポートが使用中なら前のサーバーを止めます。

画面には、会話の一覧、質問欄、「会話をリセット」があります。

観察1：履歴の有無

チャットアプリで、次の2つを続けて送ります。

1. 「私の名前は佐藤です。」
2. 「私の名前は？」

→ 答えられる。画面下の「送った履歴」の件数を見ます。

「会話をリセット」を押し、「私の名前は？」だけを送ります。

→ 答えられない。履歴を送っていないからです。

観察2：資料なし

チャットアプリで「この授業で使うモデルは何？」と送ります。

- 具体的なモデル名を答えたか、分からないと答えたかを記録
- 授業資料は何も渡していないことを確認

具体的な答えが返った場合も、後でスライドと照合します。
自然な文章でも、根拠がなければ推測かもしれません。

完成アプリの起動：キーワード検索アプリ

チャットアプリをCtrl+Cで止め、同じターミナルで起動します。

```
cd ../keyword-app
python init_db.py
python main.py
```

`init_db.py` は、`data/session01.md`（今日のスライド）を
スライド1枚ずつ `documents.db` に保存します。最初に1回だけ実行します。

「ポート」タブから8001番を開きます。

観察3：資料あり

キーワード検索アプリで、同じ「この授業で使うモデルは何？」を送ります。

1. 「検索語」に、質問から取り出された単語を見る
2. 「LLMに渡した資料」から、該当するスライドを探す
3. 回答に付いた資料IDと、その本文を照合する

資料を渡しても、回答が正しいとは限りません。本文と見比べます。

観察4：資料にない質問

キーワード検索アプリで「来年度の試験日は何月何日？」と送ります。

今日のスライドに、試験日は書かれていません。
ただし、この質問文はこのスライドに書かれているので、検索ではこのスライドが見つかります。
資料が見つかることと、資料に答えがあることは別です。

起こりうること	意味
「資料からは確認できません」	資料は渡したが、根拠が無いと判断した
具体的な日付を答えた	根拠のない断定。本文と照合して確かめる
「関係する資料が見つかりませんでした」	検索で0件。APIを呼んでいない

答えのない質問での振る舞いも、アプリを評価する材料です。

比較結果の共有

観察	見たこと
1 履歴の有無	リセット前後で、名前を答えられたか
2 資料なし	モデル名を断定したか、分からないと答えたか
3 資料あり	資料IDと本文が一致したか
4 資料にない質問	0件だったか、保留したか、断定したか

予想と違った点を、隣の人に1つ説明します。

終了時の手順

1. サーバーをCtrl+Cで停止する
2. `unset OPENAI_API_KEY` を実行する
3. ターミナルを閉じる

`documents.db` は `.gitignore` に入っているので、コミットされません。
次回以降も、起動のたびにキーを設定します。

確認問題

1. 自然な文章なら、その内容は必ず正しい？
2. 資料を質問に添えたら、モデルを学習し直したことになる？
3. チャットアプリが前の話を踏まえて答えられるのは、モデルが覚えているから？
4. このアプリで資料を選んで入力に含めるのは誰？

提出物と次回

提出：Forkした自分のリポジトリのURLを、フォームから提出します。

今日はコードの変更はありません。
Codespacesが起動し、2つのアプリを動かせたことを確認します。

次回はPythonから **gpt-6-luna** のAPIを呼び出します。